

Submission Notes

State Management and Data Fetching Approach

The application uses React Query for managing server-side data because session information is fetched from an external API source. React Query provides built-in caching, request lifecycle management, loading states, error handling, and automatic refetching capabilities. This keeps API-related logic separate from UI components and avoids manually managing asynchronous state.

For local UI states such as selected student filters and date ranges, React's built-in `useState` hook is sufficient because these values are temporary and only affect the current dashboard view. A global state-management library was not introduced because the application does not contain complex client-side shared state.

Simplifications Due to Time Constraints

The project uses JSON Server as a mock API instead of building a complete backend because the project focuses primarily on frontend engineering skills such as UI development, state management, data fetching, filtering, routing, and visualization. A production backend would add complexity around database design, authentication, APIs, and deployment, which are outside the main evaluation scope.

Authentication is implemented as a simple mock authentication flow instead of a complete token-based authentication system because the goal was to demonstrate route protection and user flow. A production implementation would require secure authentication, session management, password handling, and authorization rules.

Pagination, advanced filtering, and extensive testing were simplified because the mock dataset size was small and the project focuses on demonstrating functionality rather than optimizing for large-scale data. These features would become important when handling thousands of sessions.

The chart implementation focuses only on the required session metrics over time because the project specifically required visualization of evaluation metrics. Additional analytics such as cross-session comparisons, trend analysis, exporting reports, and advanced dashboards were not included to keep the implementation focused and deliver the core requirements within the given time frame.

Scaling to 10,000 Sessions

For handling 10,000 sessions, the current client-side filtering approach would need improvement.

The application should move filtering, sorting, and pagination responsibilities to the backend API. The API should support query parameters such as student, date range, page number, and page size.

The frontend would use server-side pagination with React Query to fetch only the required data instead of loading all sessions into memory.

Additional improvements would include database indexing on frequently searched fields, API caching, virtualized tables for large datasets, and optimized chart data aggregation. Technologies such as TanStack Table virtualization or server-side analytics processing could be introduced for better performance.